

0 Créer une table gamme et la remplir des données

créer Métier Gamme

```
<?php
class Gamme
{
    private $gam_id;
    private $gam_libelle;
    public function __construct($id,$libelle)
    {
        $this->gam_id=$id;
        $this->gam_libelle=$libelle;
    }
    public function GetGamId()
    {
        return $this->gam_id;
    }
    public function GetGamLibelle()
    {
        return $this->gam_libelle;
    }
}
?>
```

puis créer modèle m_gamme

et

dans le controller menu ajouter un fill data avec la base gamme

enfin

modifier la vue menu ajoutez gamme dans le menu

euh modifiez modele produit pour triez par gamme aussi

car bien sûr faut ajoutez gamme dans produit pour les triez

Créez PRIMARY KEY (`pro\_id`),

FOREIGN KEY (`pro\_gamme`) REFERENCES `gamme`(`gam\_id`)

exemple

Alors version propre

1: après importation de la base de données

2: faire Classe Métier gamme

modèle m_gamme

modifié dans c_menu et c_consulterProduits

dans index juste faire ,\$_GET['gam']

créer nouveaux div label select mais pour gamme

pour le filtre changer dans le modele de produit le if pour ajouter la gamme

modifie la vue de la liste pour y ajouter la gamme lorsqu'elle est choisi

```
<h1 class="mb-4 text-success">Liste des produits</h1>

<?php
if (is_null($this->data['laCategorie']) && is_null($this->data['laGamme']))
{
    echo '<h2 class="h5 mb-4 text-muted">Tous les produits</h2>';
}
elseif (isset($this->data['laCategorie']) && is_null($this->data['laGamme']))
{
    echo '<h2 class="h5 mb-4">Catégorie : '.$this->data['laCategorie']->GetLibelle(). '</h2>';
}
elseif (isset($this->data['laGamme']) && is_null($this->data['laCategorie']))
{
    echo '<h2 class="h5 mb-4">Gamme : '.$this->data['laGamme']->GetLibelle(). '</h2>';
}
else
{
    echo '<h2 class="h5 mb-4">Catégorie : '.$this->data['laCategorie']->GetLibelle().
}
?>
```

Question 3:

étape 1 : créer dans la base de données une nouvelle table avec une id en clé primaire qui s'auto incrémente et l'email qui sera stocké
ajouté aussi dans la bdd la confirmation ou non du mail char(1)

étape 2 : mettre un bouton dans l'accueil qui mène a un form de renseignement de l'email
dans la vue accueil ajoutez href avec un forme créez un contrôleur modifiez Index pour y ajouter un case

Dans Vue renvoyez encore vers index pour dirigé vers page d'ajout de l'email abonné
Créer classe métier et dcp modèle newsletter qui seront utilisé dans la méthode du contrôleur

étape 3: créer le form avec une zone de texte sécurisé ou pas et avec bouton de confirmation qui envoie un lien de confirmation au mail
et enregistre dans la base avec une requête sql INSERT
mettre unique ou pas

étape 4 pour envoyer un mail j'utilise smtp et phpmailer que je configure dans les code je vérifie si ça fonctionne avec smtp

étape 5 j'ajoute le mail a vérifié en valeur dans le lien de confirmation pour identifier le mail

étape 6 : je récupère celui ci avec un `$_GET` a l'arrivé sur le site si le mail est bien récupéré message de confirmation + inscription succès à travers `index.php` encore ajouté un case pour ajouté une page de confirmation du lien de mail
sinon message d'erreur de confirmation + message de non inscription

Étape 1 : Créer la table dans la base de données avec `id` en clé primaire `AUTO_INCREMENT`, une colonne `email` de type `VARCHAR` (avec contrainte `UNIQUE` pour éviter les doublons), et une colonne `confirmation` en `CHAR(1)` avec une valeur par défaut à 'N'.

Étape 2 : Ajouter un lien ou un bouton dans la vue accueil pointant vers `index.php?action=inscription`, puis modifier le contrôleur principal pour inclure un `case` qui renvoie vers une vue contenant le formulaire. Créer parallèlement la classe métier `Abonne` et le modèle `NewsletterModel` qui gérera les interactions avec la base de données.

Étape 3 : Concevoir le formulaire HTML avec un champ de saisie pour l'adresse mail et un bouton de validation. À la soumission, le contrôleur appelle la méthode du modèle pour exécuter une requête SQL `INSERT INTO` afin d'enregistrer l'email avec le statut de confirmation à 'N'.

Étape 4 : Configurer l'envoi de mail dans le code en utilisant la bibliothèque PHPMailer et les paramètres SMTP (hôte, port, authentification) en testant le bon fonctionnement via l'outil SMTP4 pour intercepter les messages localement.

Étape 5 : Inclure l'adresse mail directement en paramètre dans l'URL du lien de confirmation envoyé dans le corps du message (ex: `confirm.php?email=...`) pour permettre d'identifier quel compte doit être activé.

Étape 6 : À la réception du clic sur le lien, utiliser `$_GET['email']` dans `index.php` via un nouveau `case` de redirection. Si l'email est bien récupéré et présent en base, exécuter un `UPDATE` pour passer la confirmation à 'Y' et afficher un message de succès d'inscription, sinon afficher un message d'erreur de confirmation.